```
定义 整型 年龄 是 11
如果 年龄 大于 10
    输出 "长大了"
结束

对于 i 从 1 到 10
    给 总数 加上 i
结束
```

中 文 编 程 语 言

小逻辑

K I D L A N G　　v 0 . 2

> 用中文写程序，
> 让机器的每一步推理都看得见。

使用手册　　2026 · 11　　零依赖 · 单文件 · 双击即用

一、这是什么

小逻辑是一门中文编程语言。语法骨架照着 C 和 Python 来，关键字是中文的。你可以用它定义变量、写分支和循环、定义函数、操作数组和指针、读写文件——完整的一门语言，不是玩具。

它和一般编程环境最大的区别是：**程序不是「跑一下就出结果」，而是「一步一步演给你看」**。每执行一句，你能看到当前走到第几行、哪个变量的值变了、程序的每一步推理在做什么。

不依赖缩进

语句边界靠关键字，缩进乱了、整段挤一行都照跑。

没有 = 和 ==

赋值说「把 x 改成」，比较说「等于」，从语法层消灭混淆。

每步可回看

行高亮 + 变量盒子 + 中文旁白，程序不再是个黑盒。

能写真正的算法

递归、二维数组、指针、文件，该有的都有。

三步上手

1. **打开**——双击「小逻辑」图标。不需要安装任何东西。
2. **选个示例**——**顶部下拉框**里有九个现成程序，从「类型与变量」到「文件读写」。
3. **点「运行完」**——看它一步步跑，或者点「下一步」一句一句走。

> 想立刻写第一句？清空左边，敲：`输出 "你好"`　然后按 **Ctrl + Enter**。

二、界面导览

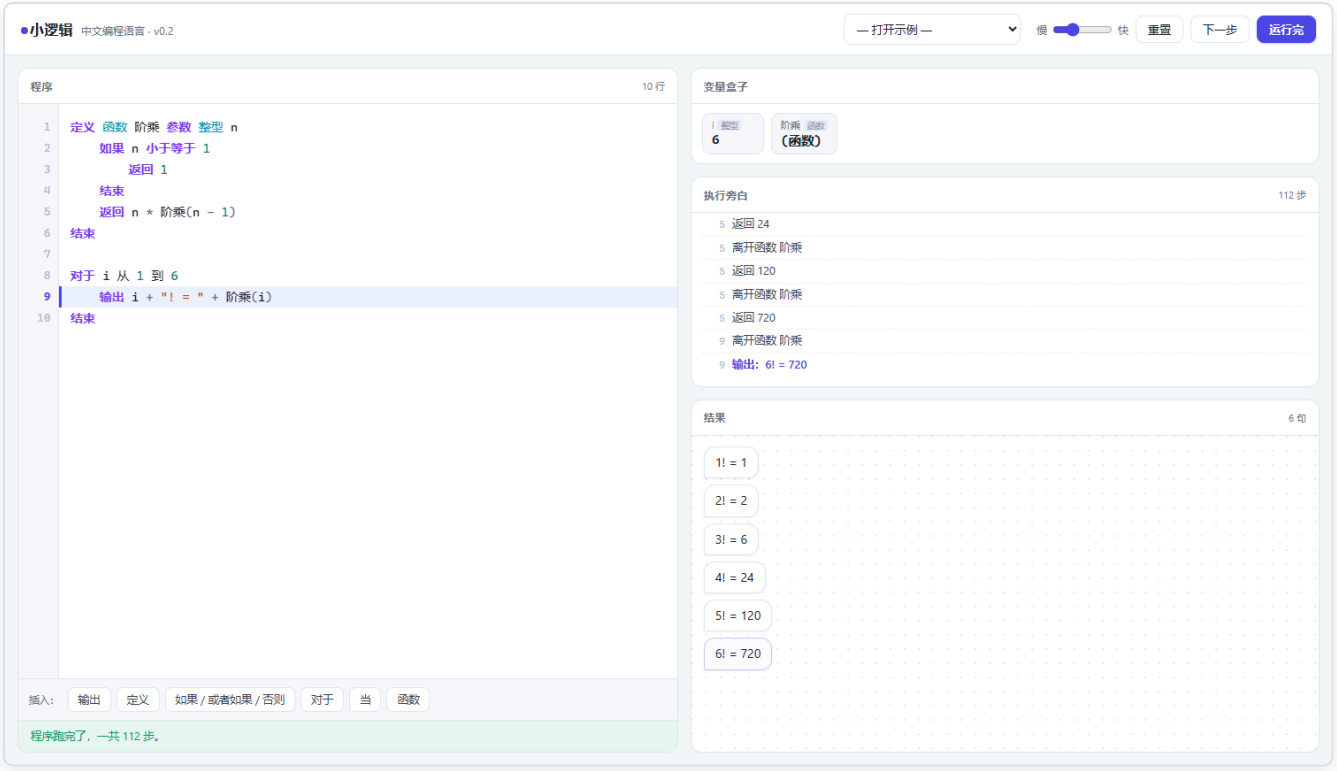


图 1　运行「三种循环」示例之后的界面

① 功能栏（最上方）

左边是示例下拉和速度滑块，右边三个按钮：**重置**回到最初，**下一步**走一句，**运行完**一口气跑到底（步数少时带动画，步数多时直接出结果）。

② 编程区（左侧）

带语法高亮：关键字紫、类型青、数字绿、文字橙、注释灰。左侧行号会标出**当前执行到第几行**，语法出错时那一行变红。底部一排「插入」按钮可以快速塞入骨架，点「如果」就自动补上 `如果` … `结束`。

③ 右上：变量盒子 + 执行旁白

变量盒子显示当前所有变量，名字右边的小灰标是类型。值刚变过的那张卡会放大闪一下，一眼看出这句改的是谁。**执行旁白**用中文解释每一步：走了哪个分支、循环到第几圈、哪个变量从几变成几。

④ 右下：结果

程序用 `输出` 说出来的内容，一句一个气泡，最新那条描边高亮。有文件操作时底部会多一个文件卡片，显示虚拟文件里的真实内容。

> **双击结果区**可以打开「猜结果」：先写你觉得会输出什么，再看答案。这是练逻辑最有用的一招。

三、类型与变量

3.1 定义变量

用 `定义` 起头，类型写在名字前面：

```
定义 整型 年龄 是 11
定义 浮点型 身高 是 1.52
定义 布尔型 通过 是 真
定义 字符型 等级 是 'A'
定义 字符串 名字 是 "小明"
```

类型可以不写，会按值的类型自动定下来：`定义 身高 是 1.52` 自动成为浮点型。

3.2 八种类型

类型	写出来长什么样	说明
`整型`	`42`	整数，不能存小数
`浮点型`	`3.14`	小数
`布尔型`	`真` `假`	只有两个值
`字符型`	`'A'`	单引号，只能一个字符
`字符串`	`"你好"`	双引号，一串文字
`数组`	`[1, 2, 3]`	一组值，下标从 0 开始
`指针`	`&分数`	指向另一个变量
`文件`	`打开文件 "a.txt" 为 写`	文件句柄

3.3 改值与转换

```
把 分数 改成 95          # 重新赋值
给 分数 加上 5           # 累加，等价于 分数 = 分数 + 5

整型(3.9)        # → 3
浮点型("2.5")    # → 2.5
字符串(123)      # → "123"
```

累加只有 `加上 / 减去 / 乘以 / 除以` 四种，写清了动词，比 `+=` 好懂。

> **类型检查是宽松的。**整型变量收到 `"42"` 会自动转成 42；但收到 `3.5` 会直接拒绝，告诉你「「年龄」是整型，不能放小数 11.5」。

四、运算与分支

4.1 四则运算（语义同 Python）

写法	结果	说明
`7 / 2`	`3.5`	真除法，保留小数
`7 // 2`	`3`	整除，向下取整
`7 % 3`	`1`	取余（符号跟除数走）
`2 ** 10`	`1024`	乘方
`"ab" * 3`	`ababab`	文字重复
`"你" + "好"`	`你好`	文字拼接

中文写法也认：`加 减 乘 除 整除 取余 乘方`。全角符号 `＋ － × ÷` 也认。

4.2 比较与逻辑

类别	写法
比较	`大于　小于　等于　不等于　大于等于　小于等于`
逻辑	`与　或　非`

> **逻辑运算只接受布尔值。** `如果 1 与 真` 会报错，必须写清楚比较：`如果 分数 大于 60 与 姓名 等于 "小明"`。C 语言里「非零即真」是初学者出错的重灾区，这里直接堵掉。

4.3 分支

```
如果 分数 大于等于 90
    输出 "优秀"
或者如果 分数 大于等于 80
    输出 "良好"
或者如果 分数 大于等于 60
    输出 "及格"
否则
    输出 "要加油"
结束
```

`或者如果` 就是别的语言里的 `else if` / `elif`。命中第一个成立的分支后，后面的不再判断。`或者如果` 和 `否则` 都可以省略。

五、三种循环

按「你知不知道要跑几次」来选。

对于 — 知道要跑几次

```
对于 i 从 1 到 10
    输出 i
结束

对于 i 从 1 到 10 步长 3    # 1 4 7 10
```

当 — 条件成立就一直跑

```
当 n 小于等于 3
    输出 n
    给 n 加上 1
结束
```

做...结束当 — 先跑一遍再说

```
做
    输出 "至少执行一次"
结束当 n 小于 3
```

> **为什么是「结束当」而不是单独的「当」？**
> 做 的循环体里完全可能再套一个 当 循环。这时单独一个 当 分不清是内层循环的开头还是 do-while 的结尾。 结束当 一个词把「收尾」和「条件」都说清楚了，没有歧义。

循环里用 `停` 可以直接跳出来。

循环在旁白里长什么样

这是「当 n 小于等于 3」那个循环实际跑出来的旁白记录。每一圈的开始、变量怎么变、什么时候停，全都写着：

```
当 [n 小于等于 3] → 成立，第 1 圈      n = 1
输出：1
n：1 → 2
当 [n 小于等于 3] → 成立，第 2 圈      n = 2
输出：2
n：2 → 3
当 [n 小于等于 3] → 成立，第 3 圈      n = 3
输出：3
n：3 → 4
当 [n 小于等于 3] → 不成立，循环结束
```

挑哪个？

场景	用哪个
遍历 1 到 100	`对于`
「直到算出答案为止」	`当`
「先做一次，再看要不要继续」（比如读文件读第一行）	`做…结束当`

六、定义函数

```
定义 函数 阶乘 参数 整型 n
    如果 n 小于等于 1
        返回 1
    结束
    返回 n * 阶乘(n - 1)
结束

输出 阶乘(5)          # 120
```

参数可以标类型也可以不标。没有 `返回` 的函数返回空。函数能看到定义处的变量，看不到调用处的（词法作用域），递归正常工作。

递归是可见的

这是小逻辑最有意思的地方：递归调用不会糊成一团，每一步「进入函数 / 条件判断 / 返回 / 离开函数」都会出现在旁白里。跑一遍 `阶乘(3)`，你会看到 16 步清晰的进出记录：

```
1  定义函数 阶乘(n)
2  进入函数 阶乘(3)
3  如果 [n 小于等于 1] → 不成立
4  所有条件都不成立，跳过
5  进入函数 阶乘(2)
...
10 返回 1
11 离开函数 阶乘
12 返回 2
13 离开函数 阶乘
14 返回 6
15 离开函数 阶乘
16 输出：6
```

把函数当语句用

```
定义 函数 喊 参数 文字
    输出 文字
结束

喊("喂")      # 调用本身就可以是一句话
```

七、数组与指针

7.1 数组

```
定义 数组 分数 是 [92, 78, 85, 60]
定义 数组 空格子 长度 5        # 5 个 0

输出 分数[0]                  # 92
把 分数[1] 改成 100
输出 长度 分数                # 4
```

支持多维：`定义 数组 m 是 [[1,2],[3,4]]`，取 `m[1][0]` 得 3。下标越界会报错并说明原因：*数组只有 4 个元素，取不到第 9 个 —— 下标从 0 开始数。*

遍历数组的常见写法

```
定义 整型 和 是 0
对于 i 从 0 到 长度 分数 - 1
    给 和 加上 分数[i]
结束
```

7.2 指针

```
定义 整型 分数 是 90
定义 指针 p 是 &分数        # 也可以写：定义 指针 p 指向 分数

输出 *p                     # 顺着指针读到 90
把 *p 改成 100              # 通过指针改，分数变成 100
输出 地址 分数               # 0x1000
```

`&` 取地址、`*` 解引用，和 C 一样。指针记住的是「哪个作用域里的哪个变量」，所以能穿进函数里改外层的变量。

> **指针最容易犯的错：**以为 `定义 指针 p 是 分数` 能让 p 指向分数。必须写 `&分数`，因为指针存的是「分数的地址」而不是分数的值。

八、字符串：索引与切片

文字的用法和 Python 一样。先把一句话存起来：

```
定义 字符串 s 是 "Hello"
```

8.1 索引：取一个字

```
输出 s[0]       # H      从 0 开始数
输出 s[1]       # e
输出 s[-1]      # o      负数从末尾数
输出 s[-2]      # l
```

越界会报错，并且告诉你一共有几个字：*这段文字只有 5 个字符，取不到第 9 个*。

8.2 切片：取一段

写法是 `起点:终点:步长`，**终点不含在内**，而且三个都可以省略。

```
输出 s[0:2]     # He        取第 0、1 个，不含第 2 个
输出 s[2:]      # llo       从第 2 个到末尾
输出 s[:2]      # He        从头到第 2 个
输出 s[:]       # Hello     整份复制
输出 s[::2]     # Hlo       每隔一个取
输出 s[::-1]    # olleH     步长为负 = 倒过来
输出 s[-2:]     # lo        最后两个
```

写法	意思	举例（s = "Hello"）
`s[a:b]`	第 a 到第 b-1 个	`s[1:3]` → `el`
`s[a:]`	第 a 个往后到底	`s[3:]` → `lo`
`s[:b]`	从头到第 b-1 个	`s[:3]` → `Hel`
`s[::n]`	每隔 n-1 个取一个	`s[::2]` → `Hlo`
`s[::-1]`	倒过来	`olleH`

> **切片不报越界。**写 `s[0:99]` 只会给你整段，不会出错——这和索引不一样。索引越界是真的错，切片越界只是「取不到那么多」。

8.3 数组用同一套

```
定义 数组 a 是 [10, 20, 30, 40]
输出 a[1:3]     # [20, 30]
输出 a[-1]      # 40
输出 a[::-1]    # [40, 30, 20, 10]
```

九、字符串：方法

方法用点号调用，可以一直往上接：

```
输出 "abc".大写()                    # ABC
输出 "  x  ".去空白()                # x
输出 "a,b,c".分割(",")[1]           # b
输出 "1,2,3".分割(",").长度()       # 3
```

9.1 字符串方法

方法	作用
`长度()`	几个字
`大写()` `小写()`	全部转大小写
`首字母大写()`	只有第一个字母大写
`去空白()` `去左空白()` `去右空白()`	去掉首尾的空白
`查找(子)`	子串在第几位，找不到给 -1
`替换(旧, 新)`	替换全部，可加第三个参数限制次数
`分割(分隔符)`	切成数组；不填就按空白切
`开头是(前缀)` `结尾是(后缀)`	判断，给真或假
`包含(子)` `出现次数(子)`	有没有、有几次
`补左(宽, 填)` `补右` `居中`	对齐用，比如 `"7".补左(3, "0")` 得 `007`
`反转()`	倒过来，同 `s[::-1]`

9.2 数组方法

方法	作用
`合并(分隔符)`	拼成一段文字：`[1,2,3].合并("-")` → `1-2-3`
`反转()` `排序()` `降序排序()`	返回新数组，不改原来那个
`查找(值)` `包含(值)` `求和()`	在第几位 / 有没有 / 加起来

一个完整的例子：整理一句话

```
定义 字符串 句子 是 "  小逻辑 是 一门 中文 语言  "
定义 字符串 干净 是 句子.去空白()

输出 "长度：" + 干净.长度()
输出 "前三个字：" + 干净[0:3]
输出 "倒过来：" + 干净[::-1]

定义 数组 词 是 干净.分割()
输出 "一共 " + 词.长度() + " 个词：" + 词.合并(" / ")
```

方法名不是关键字——`大写`、`查找`、`反转` 照样能当变量名，只有跟在点号后面才算方法。

十、文件

```
定义 文件 f 是 打开文件 "成绩.txt" 为 写
写 f "小明 92"
写 f "小红 88"
关闭 f

定义 文件 g 是 打开文件 "成绩.txt" 为 读
定义 字符串 行 是 读 g
当 行 不等于 ""
    输出 行
    把 行 改成 读 g
结束
关闭 g
```

三种打开模式

模式	行为
`读`	每次 `读` 返回下一行，读到末尾返回空字符串
`写`	打开时清空文件，之后每次 `写` 追加一行
`追加`	不清空，直接在末尾续写

> **文件是虚拟的。**程序跑在内存里，不会碰到你真实的硬盘文件。写入的内容显示在右下结果区的底部，关掉重开就没了——这样怎么折腾都安全。

读文件的标准姿势

「读一行 → 判断是不是空 → 处理 → 再读一行」这个循环，几乎所有的逐行处理都是这么写的。注意 `把 行 改成 读 g` 这一句不能漏，否则会永远读第一行，死循环。

换一种写法：先读再判断（do-while）

```
做
    定义 字符串 行 是 读 g
    如果 行 不等于 ""
        输出 行
    结束
结束当 行 不等于 ""
```

两种都能跑，前者更常见。

十一、完整例子：判断质数

一个把类型、循环、分支、函数、取余全用上的程序。把 n 换成任何数都能判断。

```
定义 函数 是质数 参数 整型 n
    如果 n 小于 2
        返回 假
    结束

    定义 整型 因子 是 2
    当 因子 乘 因子 小于等于 n
        如果 n % 因子 等于 0
            返回 假
        结束
        给 因子 加上 1
    结束

    返回 真
结束

对于 i 从 2 到 30
    如果 是质数(i)
        输出 i
    结束
结束
```

值得注意的三处

- **因子 乘 因子 小于等于 n**——只需要试到 √n。这是算法优化，比试到 n 快得多。
- **提前返回**——一发现能整除就 `返回 假`，不用跑完整个循环。
- **如果是质数(i)**——函数调用可以直接当条件用，不必先存进变量。

跑起来看什么

点「运行完」，然后按「重置」，再连点几次「下一步」。你会看到：进入函数 → 定义因子 → 检查循环条件 → 判断能否整除 → 返回。整个过程像慢动作回放，右边的变量盒子同步跳动，旁白一行一行往下走。

这就是它和普通编程环境最大的区别：**你不是在猜程序怎么跑，你是在看它怎么跑。**

十二、报错怎么读

报错是给人看的，不是给机器看的。每一条都告诉你：**第几行、错在哪、下一步做什么。**

你写的	它说什么
`定义 整型 a 是 3.5`	「a」是整型，不能放小数 3.5 —— 想放小数请用「定义 浮点型」
`定义 字符型 c 是 'AB'`	字符型只能放一个字符，你放了 2 个 —— 想要一串文字请用双引号
`输出 a[9]`（只有 4 个）	数组只有 4 个元素，取不到第 9 个 —— 下标从 0 开始数
`a = 1`	这个语言里不用「=」—— 定义一个值用「定义 名字 是 值」；比较两个值用「等于」
忘了写 `结束`	少了「结束」—— 「如果 / 当 / 对于 / 定义 函数」都需要收尾
函数参数给多了	函数 f 需要 1 个参数，你给了 2 个
递归停不下来	函数调用太深了，可能是无限递归

> 语法错误会实时检查：你打字的时候那一行就变红了，不用等运行才知道。

十一、快捷键与小技巧

快捷键	效果
`F9`	运行完（一口气跑到底）
`F8`	下一步（一句一句走）
`F5`	重置（回到最初状态）
`F10`	猜结果
`Ctrl + Enter`	同 F9
`Tab`	插入 4 个空格
双击结果区	同 F10

写代码时的三个习惯

- **变量名随便用中文。**「总分」「最高分」「是不是质数」都能当名字，代码读起来像话。
- **先用「下一步」走一遍再运行完。**看清楚每一步，比盯着结果猜有用得多。
- **写完先猜结果再按运行。**双击结果区写下你的预测——猜对了说明你真懂了。

> **版本。**v0.2 支持类型、三种循环、函数、数组、指针、文件。已知没有：结构体/类、字符串的子串查找替换、switch。实现约 1500 行，零依赖。